

Kursal: A peer to peer encrypted messenger

Erik LP.
Kodeur_Kubik
kubik@kursal.chat

Armand H.
Arlo
arlo@kursal.chat

Abstract

Kursal proposes an end-to-end encrypted, peer-to-peer protocol for exchanging messages and files over a decentralized network. It uses Signal's encryption methods combined with libp2p's protocols to achieve a fully decentralized setup. The network relies on volunteer nodes: the more nodes that are connected, the more robust the network gets. Any node can offer itself as a public relay to help others connect across NATs or behind firewalls. Kursal's objective is to combine security, decentralization and usability into one unified application.

Version: 1.1

Date: April 13, 2026

Website: <https://kursal.chat/>

1. Introduction

Messaging platforms have become the most widely adopted systems for online communication, file transfers and calls. However, commonly used messengers either do not encrypt messages on their servers or lack transparency. Others compromise user privacy by collecting private user data. Even transparent systems are vulnerable. Governments could still access centralized hosting servers, block or filter content, censor users, or spy on them. This is where Kursal steps in as a potential solution: all messages, files, and calls are end-to-end encrypted and transit over a decentralized network hosted by volunteers.

1.1. Motivation

Growing centralization in digital communication threatens user privacy and autonomy. Proposals like the European Union's Chat Control initiative to scan user messages for harmful content highlight the risk of surveillance becoming normalized under the guise of safety. Kursal aims to demonstrate a different path: a fully decentralized, peer-to-peer protocol for private communication based on modern cryptography. We believe privacy and security can coexist through open, verifiable, and community-driven technology. We do not endorse crimes but believe in non-privacy-intrusive ways of countering them. Our goal is to demonstrate that decentralized peer-to-peer encrypted messaging can achieve the same

level of usability and reliability as centralized platforms.

2. First Contact

There are three ways for two peers "Alice" and "Bob" to initiate communication over the Kursal network. Each method has its advantages and disadvantages:

- **One-Time Password:** Easy to share. Takes time to publish on the network, is single-use and requires computation.
- **Long-Term Code:** Multiple usage code that requires no computation. File size of 6 KB that must be transferred via an external channel.
- **Nearby Share:** Easy to share. Devices must be on the same Wi-Fi network (which must support mDNS) to initiate the connection.

2.1. With a One-Time Password

Alice generates a one-time password (referred to as OTP) composed of 8 random words. The interface can also display this code as a QR-code that can be scanned by the other user. The word dataset partially comes from <https://www.eff.org/dice> and we filtered and added words, resulting in a total of around 11 080 words. Alice computes the `argon2id` [1] hash of this OTP (referred to as `OTP_H`). For benchmarks and information about this hashing configuration, refer to Section 6.2. An initial `PQXDH` [2] payload is encrypted with `argon2` and `AES-256` (with an empty salt). It is then published to the distributed hash table (DHT) [3] under the hash `OTP_H` and `argon2` encrypted value (`PQXDH_payload`, `Peer ID`, `pubkey`, `relays`) with the password `OTP`. The

DHT entry expires and is automatically removed after 10 minutes. The public sign key (pubkey) is a public Dilithium-5 [4] key that will later prove the integrity of messages. The relays entry contains addresses on which Alice can be contacted (direct and relays). Bob may now fetch the corresponding DHT entry by first hashing the OTP he received from an external source. He can then complete the PQXDH protocol and initiate a Double Ratchet [5] extended by the ML-KEM Braid Protocol [6]. For Alice to also complete the PQXDH protocol, Bob must send the final PQXDH payload, his public signing key, his public listening addresses (like Alice’s relays) and a Double Ratchet initial message through a relay.

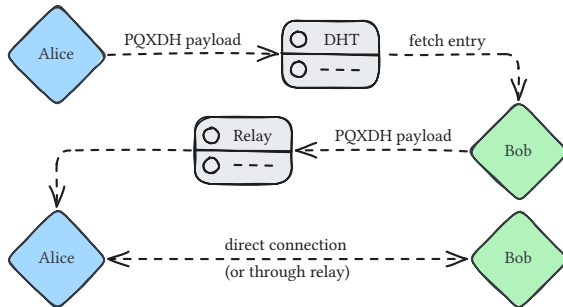


Figure 1: Direct contact with an OTP

2.2. With a Long-Term Code

Exchanging individual one-time passwords can be time-consuming. To simplify long-term contact management, Alice creates an object containing her peer ID, an initial PQXDH payload [2], an ephemeral public key, a public Dilithium-5 [4] signing key as well as public addresses to reach Alice (direct IP or via relays). The PQXDH payload does not contain a one-time pre-key as this payload can be used multiple times by different users. The ephemeral key is kept in cache by Alice and deleted once the Long-Term Code (referred to as LTC) is renewed or invalidated. This object can then be encoded into a “.kursal” file (which is approximately 6 KB in size). After receiving the file, Bob decodes this object and completes the PQXDH protocol by sending the PQXDH payload, his public signing key and his public listening addresses to Alice. They can initiate a Double Ratchet [5] extended by the ML-KEM Braid Protocol [6] as described in Section 2.1.

2.3. Nearby Share

The Nearby Share process allows devices on the same local network to discover each other via multicast DNS (mDNS). To protect user privacy and prevent unsolicited key exchanges, this is a two step process:

First, the discovery phase: devices broadcast a beacon containing a randomly generated ephemeral “adjective-animal” username (e.g., “Orange Mouse”). Users can identify each other using these temporary names.

Second, the handshake phase: one user initiates a connection, and the other must explicitly accept the request prompt. After mutual consent, the public keys and initial PQXDH payloads [2] are exchanged. This process could be extended with Bluetooth, but is not implemented in our first version of Kursal.

2.4. Security Code

Each security code is unique to each contact and should be verified as soon as the communication is established. It should match on both sides and if not, it very likely means a man-in-the-middle attack is happening. Security codes should **not** be verified by sending them over the chat because in the case of a man-in-the-middle attack, the attacker could impersonate this code as well.

It is computed by applying a key derivation function to the identity keys and signing keys of both parties in a deterministic way in order to obtain the same result on both sides. Both public identity keys and the two Dilithium-5 public signing keys are sorted and concatenated together. Those four concatenated keys are then hashed with the SHA-256 algorithm. The output is then formatted as 8 groups of 4 digits from the SHA-256 and displayed on the user interface.

3. Continuous Communication

Kursal is based on `rust libp2p’s swarm` architecture. It supports both TCP and QUIC protocols and automatically finds the optimal networking solution. Each connection initially starts from a relay and will, if possible, be upgraded using `DCUTR` (Direct Connection Upgrade Through Relay). This upgrade can be a direct peer-to-peer

connection or both peers can attempt a NAT traversal using hole punching. If both of those fail, the connection is maintained through the relay. Each node is identified by its peer ID, which rotates regularly to prevent tracking.

Each message receives a reply containing the Dilithium-5 [4] signature of that same encrypted message to prove authenticity. This signature should be verified, and the message should not be marked as delivered until Alice’s signed reply is received.



Figure 2: Signed reply of a received message

4. Data Streams

In order to transfer a large amount or a continuous stream of data like file transfers and calls, Kursal cannot encrypt each packet with Double Ratchet as this method would be too computationally intensive. Therefore, each time a data stream is about to happen (calls, file transfers, ...), both agree on a random, temporary key to encrypt all following packets using the AES-GCM [7] protocol. This allows lightweight encryption for one-time transfers. Each peer generates a random 32-byte key that is sent to the other via Double Ratchet. Both peers derive a shared key by passing the concatenation of their two keys through HKDF-SHA256. This ensures a cryptographically secure mix that cannot be guessed by compromising one of the original keys.

5. Implementation

We propose a Kursal implementation written in Rust available at <https://github.com/KursalChat/Kursal> under the [GNU AGPLv3](#) license, available on Linux, macOS and Windows. Our implementation uses Signal’s [libsignal](#) and libp2p’s [rust-libp2p](#) libraries.

We plan to improving this initial version over time by adding offline messaging support, multi-device support, groups and broadcasting channels. These features are listed on our [TODO list](#) and will be implemented over time.

Offline messaging is not yet implemented, as mentioned earlier. However, because peer IDs rotate periodically, contacts may lose track of each other if a node rotates its identity while either party is offline. To address this, Kursal implements a DHT rendezvous fallback mechanism. When a node rotates its transport identity, it constructs a rendezvous record containing its new peer ID and current reachable addresses, signs it with the user’s permanent Dilithium-5 signing key, and publishes it to the Kademlia DHT. This also happens on a daily basis. When a node comes back online and fails to reach a contact through the usual route, it queries the DHT using that contact’s public identity key to retrieve their latest routing information, allowing peers to rediscover each other and reconnect seamlessly.

6. Risk Mitigation

6.1. Operating System

If the operating system itself is compromised, Kursal cannot guarantee the integrity of messages. All encryption keys, which are stored in the [keychain](#) of the computer could be accessed by malware or by the operating system itself. It is up to the user to choose a trusted operating system and keep it secure.

6.2. DHT Brute Forcing

To evaluate possible risks of pre-calculating every possible OTP and its corresponding hash, we can estimate how long it would take to compute. Given a computer hashing speed of $CS = 1000$ hashes per second and per device, a time period of 1000 years, which approximates to $T \approx 3 * 10^{10}$ seconds. The number of possible combinations of the OTP is $P = 11000^8 = 2.14 * 10^{32}$. We can approximate the number of devices D needed with this hashing speed to calculate every possibility:

$$D = \frac{P}{CS * T} \approx 7.1 * 10^{18} \text{ devices}$$

This means that in order to intercept 0.1% of the DHT entries, one would need to compute at 1000 hashes per second for 1000 years with $7.1 * 10^{15}$ devices. And for one interception in a million,

it drops to $7.1 * 10^{12}$ devices for the next 1000 years at 1000 hashes per second. For more details about what DHT interceptions lead to, refer to Section 6.3.

The Argon2id configuration used is configured to take a significant amount of time on modern computers. We are therefore well under the theoretical 1000 hashes per second, making brute-forcing impractical and future-proof. Here are the results of the hashing benchmark (Kursal uses those exact settings) with: memory = 256 MiB, iterations = 2, parallelism = 1. The high memory limit prevents massive parallelization.

CPU	Time per iteration	Iter. per sec.
Intel Core i5-14600kf	709.5 ms 35.6 ms (threaded)	28.07
MacBook Pro M5	376.2 ms 37.7 ms (threaded)	26.53
Intel Core i5-12400F	629.3ms 52.6ms (threaded)	19.01
Intel Core i5-10310U	2234ms 280ms (threaded)	3.57

Table 1: Hashing Benchmark: 1000 iterations

The assumed value of CS = 1000 hashes per second is an extremely high speed that is not reached by modern computers. It is therefore theoretically possible to brute force the DHT, but very unlikely. If an attack of some sort happens, the Argon2id salt could be changed from null to a random one and the attackers would have to recalculate all the possibilities.

6.3. DHT Interception

Intercepting the DHT is defined by accessing an encrypted initial PQXDH payload by finding the OTP. In a hypothetical scenario, a hacker could pre-compute all pairs of OTP and their hash and decrypt the payload. The initial payload does not contain any sensitive information as it is only made up of public keys. The hacker could attempt a man-in-the-middle attack but it could be detected by a different security code (see Section 2.4).

6.4. Malicious Relays

Relays, even malicious, will never be able to read your messages as they are encrypted. However, they can analyze your messaging patterns if those patterns are left unprotected. This is why peer IDs rotate regularly to prevent tracking your communication patterns. Additionally, relays have access to your IP address if no proxy or VPN is used.

Bibliography

- [1] A. Biryukov, D. Dinu, and D. Khovratovich, “Argon2: the memory-hard function for password hashing and other applications,” technical report, Mar. 2017. [Online]. Available: <https://github.com/P-H-C/phc-winner-argon2/blob/master/argon2-specs.pdf>
- [2] E. Kret and R. Schmidt, “The PQXDH Key Agreement Protocol,” technical report, May 2023. [Online]. Available: <https://signal.org/docs/specifications/pqxdh/>
- [3] Raulk, J. Hiesey, and M. Inden, “libp2p Kademia DHT Specification,” technical report, Dec. 2022. [Online]. Available: <https://github.com/libp2p/specs/blob/master/kad-dht/README.md>
- [4] L. Ducas *et al.*, “CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme,” technical report, Feb. 2021. [Online]. Available: <https://eprint.iacr.org/2017/633.pdf>
- [5] T. Perrin, M. Marlinspike, and R. Schmidt, “The Double Ratchet Algorithm,” technical report, Sept. 2025. [Online]. Available: <https://signal.org/docs/specifications/doubleratchet/>
- [6] R. Schmidt, “The ML-KEM Braid Protocol,” technical report, Feb. 2025. [Online]. Available: <https://signal.org/docs/specifications/mlkembraid/>
- [7] D. McGrew and K. Igoe, “AES-GCM Authenticated Encryption in the Secure Real-time Transport Protocol (SRTP),” technical report NIST Special Publication 800-38D, Oct. 2008. [Online]. Available: <https://datatracker.ietf.org/doc/rfc7714/>